

Programming for Biomedical Informatics

Revision Session

GitHub Website- <https://github.com/biomedical-informatics/pbi>

Course Website- <https://groups.inf.ed.ac.uk/teaching/pbi/>

Ian Simpson
ian.simpson@ed.ac.uk

Exam Information

Wednesday, 10th December

2:30-4:30 p.m.

Appleton Tower Concourse

Structure

Question 1 (25 marks)

and choice of

Question 2 (25 marks)

or

Question 3 (25 marks)

Question 1

- short answer questions worth 2 or 3 marks covering material from whole course

Questions 2 &3

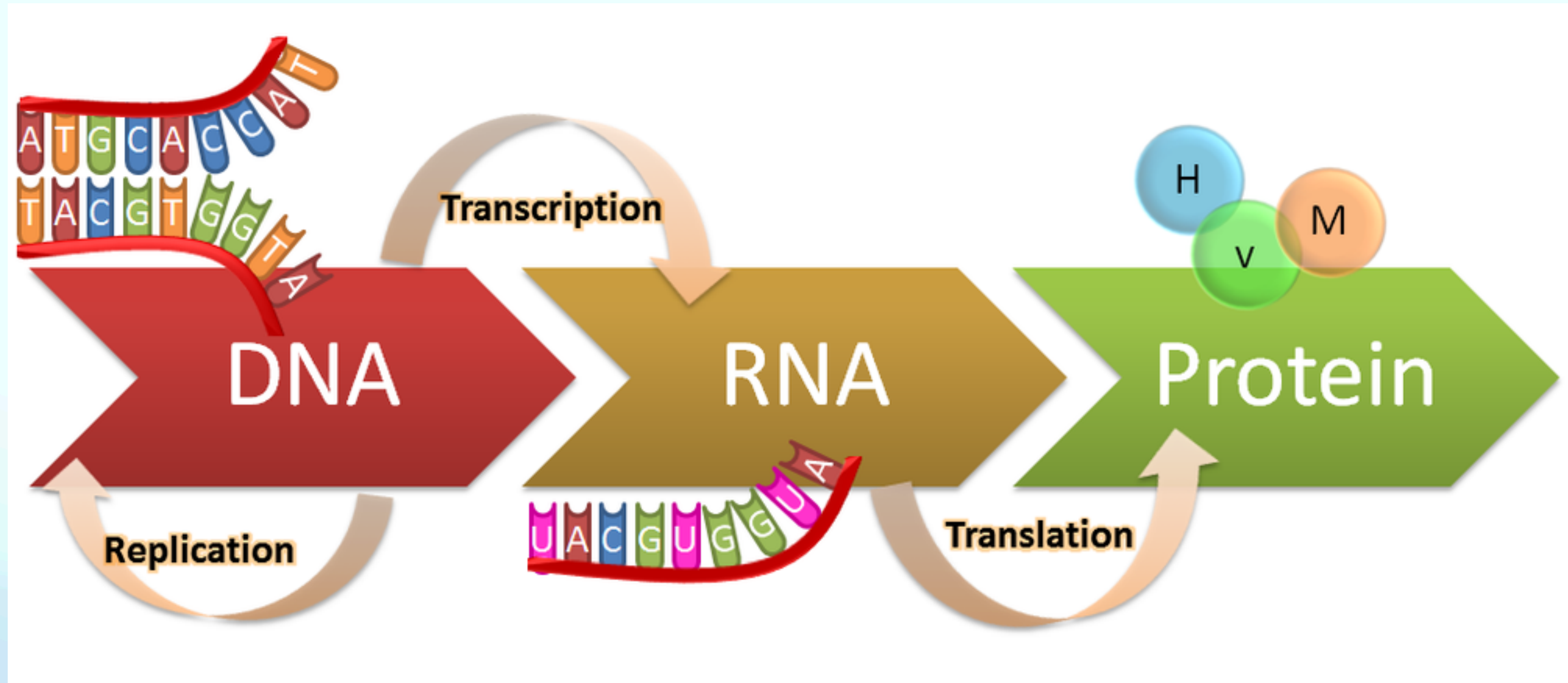
- example based (a research question/ scenario)
- Mixture of short and longer question parts 2-8 marks each
- 1x based largely on pseudo-coding
- 1x based largely on code interpretation

Overview of Lecture Materials

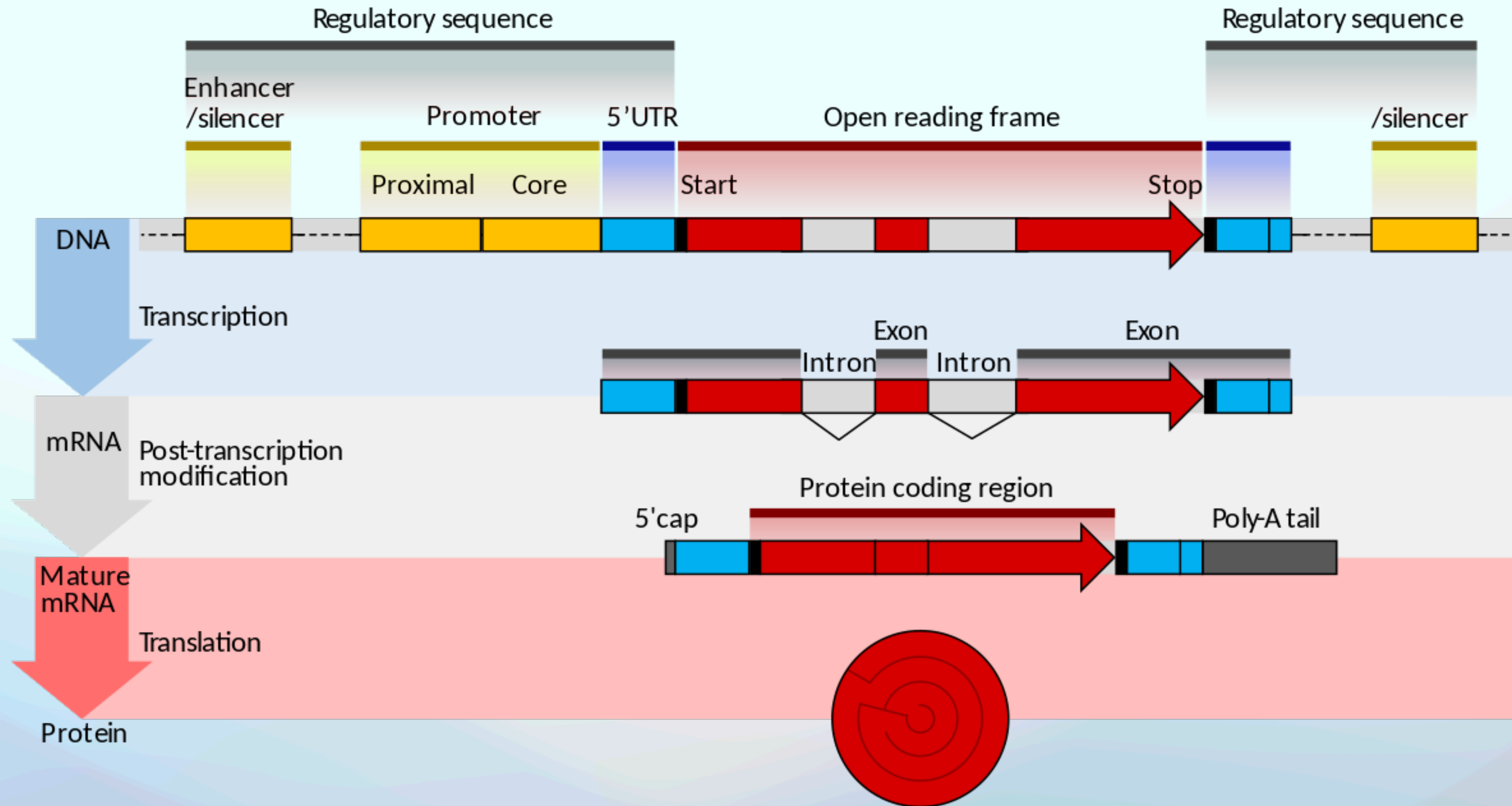
Lectures

- Week 1 - [Welcome & Getting Started](#) (lecture 1) & [Working with Notebooks & Git](#) (lecture 2)
- Week 2 - [Introduction to the Biomedical Dataverse](#) (lecture 3) & [Bulk FTP Download & APIs](#) (lecture 4)
- Week 3 - [Mapping & Harmonisation](#) (lecture 5) & [Data Integration & Summary Analysis](#) (lecture 6)
- Week 4 - [Biomedical Evidence](#) (Lecture 7) & [Mining & Analysing the Biomedical Literature](#) (lecture 8)
- Week 6 - [Measuring Gene Expression](#) (lecture 9) & [Differential Gene Expression](#) (lecture 10)
- Week 7 - [Biological Networks](#) (lecture 11) & [Network Construction Techniques](#) (lecture 12)
- Week 8 - [Essential Network Methods](#) (lecture 13) & [Network Analysis in Practice](#) (lecture 14)
- Week 9 - [Structuring Biomedical Data with Ontologies](#) (lecture 15) & [Working with Ontologies](#) (lecture 16)
- Week 10 - [Working with multiple data modalities](#) (lecture 17) & [Modelling at the patient level](#) (lecture 18)

The “Central Dogma”



The “Central Dogma”



Programmatic Bulk Download

We can use a number of Python approaches to achieve bulk download programmatically rather than having to deal with manual downloading and saving of data:

URL Fetching Libraries

- ``requests`` - ``requests.get(url).content`` followed by saving the content to file.
- ``urllib`` - ``urllib.request.urlretrieve(url, filename)`` this downloads to file.
- ``wget`` (wrapper for GNU wget) - ``wget.download(url)`` this downloads to file.

Threading of multiple downloads simultaneously

- ``Scrapy`` - large-scale web crawling and bulk data extraction, efficient for downloading data from multiple URLs by defining spiders.
- ``pycurl`` - curl interface. good for handling multiple simultaneous HTTP/FTP transfers.

Handling Authentication & Session Management

- ``requests.Session`` to persist certain parameters across requests, can handle login and cookie management for downloading from authenticated sources.
- Selenium for Web Scraping, especially for websites that load with JavaScript, can simulate a browser to fetch and download files and interact with web elements.

Application Programming Interfaces (APIs)

Abstraction - APIs provide a simple interface to more complex underlying systems. They abstract the internal workings exposing only what is necessary for the user.

Standardisation - APIs are designed with standard protocols and rules, ensuring consistency

Security - APIs enforce security, can limit access by requiring authentication, and ensure data integrity and confidentiality through encryption.

Scalability - APIs allow systems to handle increased volumes of data or interactions.

Efficiency - Data exchange through APIs can be optimised to reduce the amount of data that needs to be sent.

Documentation - APIs commonly come with comprehensive documentation that details available endpoints, data formats, and methods.

Versioning - APIs often employ version control, allowing them to introduce new features or deprecate old ones without disrupting existing code developed using them.

Customisation - APIs commonly allow the user to specify parameters associated with their query to tailor data retrieval to their specific needs.

The NCBI-NLM eUtilities

- **EInfo (database statistics)** - eutils.ncbi.nlm.nih.gov/entrez/eutils/einfo.fcgi
 - Provides the number of records indexed in each field of a given database, the date of the last update of the database, and the available links from the database to other Entrez databases.
- **ESearch (text searches)** - eutils.ncbi.nlm.nih.gov/entrez/eutils/esearch.fcgi
 - Responds to a text query with the list of matching UIDs in a given database (for later use in ESummary, EFetch or ELink), along with the term translations of the query.
- **EPost (UID uploads)** - eutils.ncbi.nlm.nih.gov/entrez/eutils/epost.fcgi
 - Accepts a list of UIDs from a given database, stores the set on the History Server, and responds with a query key and web environment for the uploaded dataset.
- **ESummary (document summary downloads)** - eutils.ncbi.nlm.nih.gov/entrez/eutils/esummary.fcgi
 - Responds to a list of UIDs from a given database with the corresponding document summaries.
- **EFetch (data record downloads)** - eutils.ncbi.nlm.nih.gov/entrez/eutils/efetch.fcgi
 - Responds to a list of UIDs in a given database with the corresponding data records in a specified format.
- **ELink (Entrez links)** - eutils.ncbi.nlm.nih.gov/entrez/eutils/elink.fcgi
 - Responds to a list of UIDs in a given database with either a list of related UIDs (and relevancy scores) in the same database or a list of linked UIDs in another Entrez database; checks for the existence of a specified link from a list of one or more UIDs; creates a hyperlink to the primary LinkOut provider for a specific UID and database, or lists LinkOut URLs and attributes for multiple UIDs.
- **EGQuery (global query)** - eutils.ncbi.nlm.nih.gov/entrez/eutils/egquery.fcgi
 - Responds to a text query with the number of records matching the query in each Entrez database.
- **ESpell (spelling suggestions)** - eutils.ncbi.nlm.nih.gov/entrez/eutils/espell.fcgi
 - Retrieves spelling suggestions for a text query in a given database.
- **ECitMatch (batch citation searching in PubMed)** - eutils.ncbi.nlm.nih.gov/entrez/eutils/ecitmatch.cgi
 - Retrieves PubMed IDs (PMIDs) corresponding to a set of input citation strings.

eUtils Mapping Review

- We used a selection of combinations of eUtils tools to recover information that allows for mapping between sources
- eSearch - can take mixed queries, typically terms or phrases e.g. gene symbols "Pax6"
- eSummary - can take identifiers returned by eSearch to recover basic summary information. This is often what you need as it includes internal numerical NCBI identifiers and basic meta-data (descriptions, names...)
- eLink - can be used to map between NCBI databases e.g. from "nucleotide" to "gene". It returns a LinkSet in which individual Link items contain the cross-mapped information (NM_12345 -> GeneID:678)
- eFetch - can be used to pull full entries. These are typically extremely long and complex records with deeply nested elements and/or long lists of features - there are options to filter these

eUtils Mapping Review

- We can use the BioPython Entrez module for more accessible use of eUtils and it can be extended to more complex implementations
- Once familiar with the basic function of the eUtils it is more flexible to begin using the requests library to make API calls using URLs that you have built
- This makes it easier to take advantage of eUtils features like “WebEnv & QueryKey” and to serialise large requests that are much more difficult to retrieve efficiently through other means
- Key to the requests approach is an understanding of how you parse XML/JSON response texts in Python to access the features you are looking for.
- Typically this requires that you first print out an XML response for a small query so that you can correctly format your XML element search.
- Once this has been perfected you can readily scale up your analysis efficiently.

eLink “link-names”

Begin with

Q

Search

<>

Don

Entrez Link Descriptions:									
PubMed	Protein	Nucleotide	Structure	Genome	Books	CancerChromosomes	Conserved Domains	3D Domains	Gene
Genome Project	GENSAT	GEO Profiles	GEO DataSets	HomoloGene	MeSH	NCBI Site Search	OMIA	OMIM	
PMC	PopSet	Probe	PubChem BioAssay	PubChem Compound	PubChem Substance	SNP	Taxonomy	UniGene	UniSTS

Link name (used in Entrez URLs)	Label in "Links" menu	Label in "Display" menu	
assembly_assembly_diploid	Linked assembly from diploid	Linked assembly from diploid	Genome assembly that is the other (ps
assembly_bioproject	BioProject	BioProject	BioProject
assembly_bioproject	BioProject	BioProject	BioProject
assembly_bioproject	BioProject		
assembly_biosample	BioSample	pubmed_assembly	Assembly
assembly_biosample	BioSample	pubmed_assembly	Assembly
assembly_biosample	BioSample	pubmed_assembly	Assembly
assembly_genome	Genome	pubmed_bioproject	Related Project
assembly_genome	Genome	pubmed_bioproject	Related Project
assembly_genome	Genome	pubmed_bioproject	Related Project
assembly_nuccore	Nucleotide	pubmed_biosample	BioSample
assembly_nuccore_insd	Nucleotide INSDC	pubmed_biosample	BioSample
assembly_nuccore_refseq	Nucleotide RefSeq	pubmed_biosample	BioSample
assembly_nuccore_wgsmaster	WGS Master	pubmed_biosample	BioSample
assembly_nuccore_wgscontig	WGS contigs	pubmed_biosystems	BioSystems
assembly_nuccore_insd	Nucleotide INSDC	pubmed_biosystems	BioSystems
assembly_nuccore_refseq	Nucleotide RefSeq	pubmed_biosystems	BioSystems
assembly_nuccore_wgsmaster	WGS Master	pubmed_books_refs	Cited in Books
assembly_nuccore_insd	Nucleotide INSDC	pubmed_books_refs	Cited in Books
assembly_nuccore_refseq	Nucleotide RefSeq	pubmed_books_refs	Cited in Books
assembly_nuccore_wgsmaster	WGS Master	pubmed_cancerchromosomes	Cancer Chromosomes
assembly_pubmed	PubMed	pubmed_cdd	Conserved Domains
assembly_pubmed	PubMed	pubmed_cdd	Conserved Domains
assembly_pubmed	PubMed	pubmed_cdd	Conserved Domains
assembly_sra	Sra	pubmed_clinvar	ClinVar
assembly_sra	Sra	pubmed_clinvar_calculated	ClinVar (calculated)
assembly_taxonomy	Taxonomy	pubmed_clinvar	ClinVar
assembly_taxonomy	Taxonomy	pubmed_clinvar_calculated	ClinVar (calculated)
assembly_taxonomy	Taxonomy	pubmed_clinvar	ClinVar
biocollections_biosample	BioSample	pubmed_clinvar	ClinVar
biocollections_nuccore	Nucleotide	pubmed_clinvar_calculated	ClinVar (calculated)
biocollections_nuccore	Nucleotide	pubmed_dbvar	dbVar
biocollections_nucgss	GSS	pubmed_dbvar	dbVar
biocollections_nucgss	GSS	pubmed_dbvar	dbVar
biocollections_popset	PopSet	pubmed_dbvar	dbVar
biocollections_popset	PopSet	pubmed_domains	3D Domains
biocollections_popset	PopSet		
biocollections_protein	Protein		
biocollections_protein	Protein	pubmed_epigenomics	Epigenomics
bioconcepts_bioconcepts_child	children	pubmed_epigenomics_experiment	Epigenomics (experiments)
bioconcepts_bioconcepts_parent	parent	pubmed_epigenomics_sample	Epigenomics (samples)
bioconcepts_bioconcepts_sibling	sibling	pubmed_epigenomics_study	Epigenomics (study)
bioconcepts_bioconcepts_child	children	pubmed_epigenomics_experiment	Epigenomics (experiments)
bioconcepts_bioconcepts_parent	parent	pubmed_epigenomics_sample	Epigenomics (samples)
bioconcepts_bioconcepts_sibling	sibling	pubmed_epigenomics_study	Epigenomics (study)
bioconcepts_bioconcepts_child	children	pubmed_gap	dbGaP
bioconcepts_bioconcepts_parent	parent	pubmed_gap	dbGaP
bioconcepts_bioconcepts_sibling	sibling	pubmed_gap	dbGaP
		pubmed_gap	dbGaP
		pubmed_gcassembly	Assembly for publication.
		pubmed_gds	GEO DataSets
		pubmed_gds	GEO DataSets
		pubmed_gds	GEO DataSets
		pubmed_gene	Gene Links
		pubmed_gene_rif	Gene (GeneRIF)
		pubmed_gene_citedinomim	Gene (OMIM)
		pubmed_gene_bookrecords	Gene (from Bookshelf)
		pubmed_gene_pmc_nucleotide	Gene (nucleotide/PMC)
		pubmed_gene	Gene
		pubmed_gene_bookrecords	Gene (from Bookshelf)

DESeq2 for Differential Gene Expression Analysis

DESeq2 is a popular tool implemented in R and Python for analysing count-based data like RNA-seq. It provides methods to test for differential expression by using negative binomial generalised linear models. The core of DESeq2's approach lies in its ability to accurately estimate variance-mean dependencies in count data, and to test for differential expression based on these estimates.

Love MI, Huber W, Anders S. Moderated estimation of fold change and dispersion for RNA-seq data with DESeq2. Genome Biol. 2014;15(12):550. doi: 10.1186/s13059-014-0550-8

- Data is input as genes (rows) x samples (columns) matrix, along with sample information (conditions, replicates etc.).
 - **Filtering:** Low count genes are removed and gross normalisation for library size and sequencing depth are made
 - **Size Factor Calculation:** each gene's count is divided by the geometric mean of the counts for that gene across all samples
 - **Median Ratio Method:** size factors are calculated by taking the median of these ratios for each sample and these are used to normalise across samples.
- Dispersion is then estimated.
 - **Gene-wise:** each gene's dispersion is estimated from its mean and variance
 - **Shrinkage:** A Bayesian method is used to shrink the gene-wise dispersion estimates to moderate the values. This is especially important when the number of replicates is small.
 - **Fitting Dispersion-Mean Relationship:** A fit is made of dispersion estimates against mean expression to balance out gene-wise estimates and provide stable measures for all genes.
- Statistical Modelling & Hypothesis Testing
 - **Model Design:** A generalised linear model is used to incorporate information about experimental design, such as treatment conditions.
 - **Wald's Test or Likelihood Ratio Test:** DESeq2 then performs a statistical test (typically Wald's test) for each gene to determine whether it is significantly differentially expressed across conditions.
 - **Multiple Testing Correction:** Methods including Benjamini-Hochberg procedure are used to adjust p-values (often presented as an FDR - False Discovery Rate).

Network - Representations

Adjacency Matrix

$$\mathbf{A} = \begin{array}{c|cccccc} & \mathbf{1} & \mathbf{2} & \mathbf{3} & \mathbf{4} & \mathbf{5} & \mathbf{6} \\ \hline \mathbf{1} & 0 & 1 & 0 & 0 & 0 & 0 \\ \mathbf{2} & 1 & 0 & 1 & 0 & 1 & 0 \\ \mathbf{3} & 0 & 1 & 0 & 1 & 0 & 0 \\ \mathbf{4} & 0 & 0 & 1 & 0 & 1 & 0 \\ \mathbf{5} & 0 & 1 & 0 & 1 & 0 & 1 \\ \mathbf{6} & 0 & 0 & 0 & 0 & 1 & 0 \end{array}$$

$$k_i = \sum_{j=1}^n A_{ij} \quad (\text{degree of node } i)$$

Degree Matrix

$$\mathbf{D} = \begin{array}{c|cccccc} & \mathbf{1} & \mathbf{2} & \mathbf{3} & \mathbf{4} & \mathbf{5} & \mathbf{6} \\ \hline \mathbf{1} & 1 & 0 & 0 & 0 & 0 & 0 \\ \mathbf{2} & 0 & 3 & 0 & 0 & 0 & 0 \\ \mathbf{3} & 0 & 0 & 2 & 0 & 0 & 0 \\ \mathbf{4} & 0 & 0 & 0 & 2 & 0 & 0 \\ \mathbf{5} & 0 & 0 & 0 & 0 & 3 & 0 \\ \mathbf{6} & 0 & 0 & 0 & 0 & 0 & 1 \end{array}$$

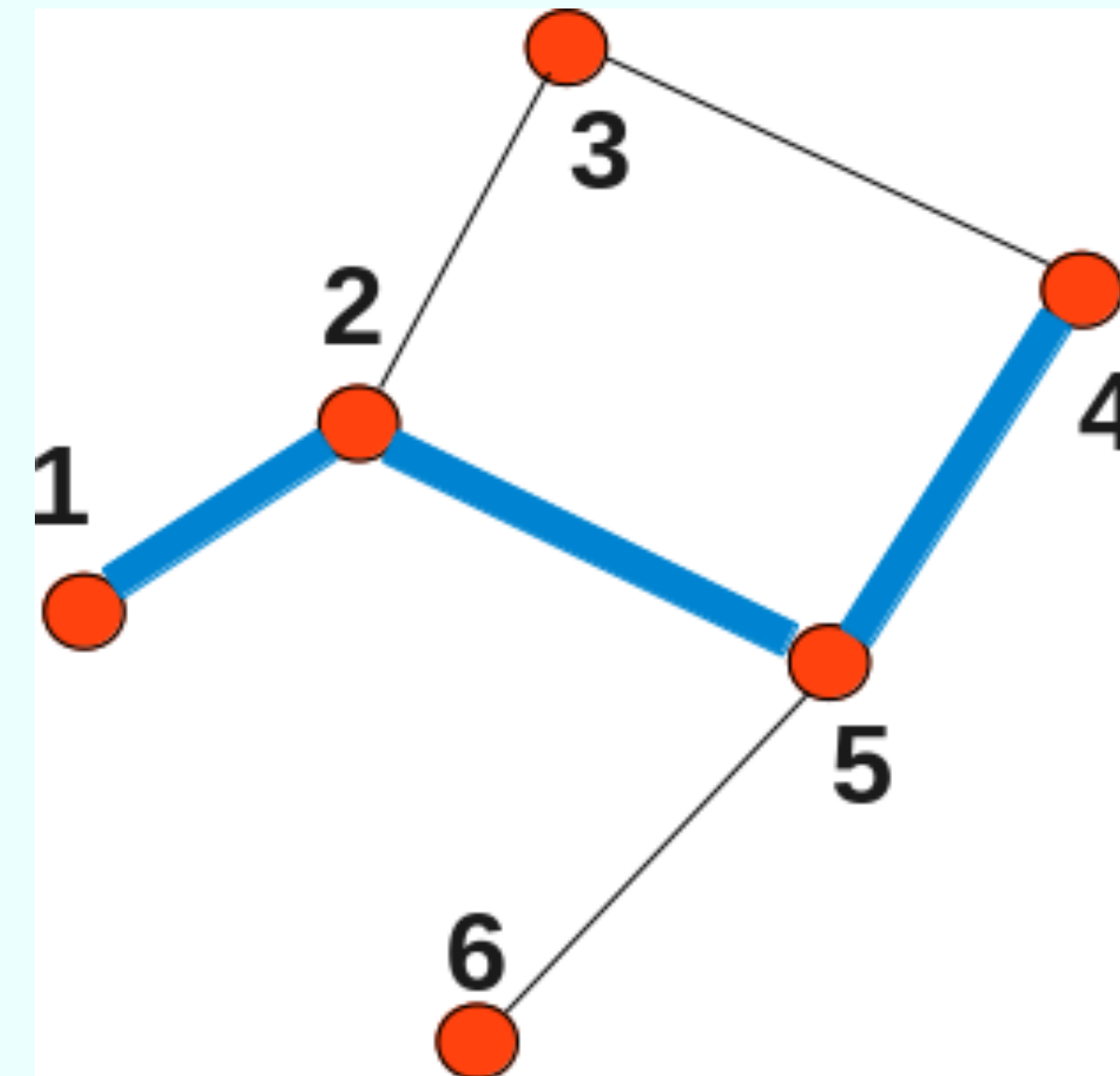
$$A_{ij} \in \mathbb{R} \quad (\text{unweighted} = 0,1)$$

$$A_{ij} = A_{ji} \quad (\text{undirected})$$

$$A_{ii} = 2 \quad (\text{if self-edge})$$

(edges in network)

$$\frac{1}{2} \sum_{i=1}^n k_i = 6$$



Laplacian Matrix

$$\mathbf{L} = \mathbf{D} - \mathbf{A} = \begin{array}{c|cccccc} & \mathbf{1} & \mathbf{2} & \mathbf{3} & \mathbf{4} & \mathbf{5} & \mathbf{6} \\ \hline \mathbf{1} & 1 & -1 & 0 & 0 & 0 & 0 \\ \mathbf{2} & -1 & 3 & -1 & 0 & -1 & 0 \\ \mathbf{3} & 0 & -1 & 2 & -1 & 0 & 0 \\ \mathbf{4} & 0 & 0 & -1 & 2 & -1 & 0 \\ \mathbf{5} & 0 & -1 & 0 & -1 & 3 & -1 \\ \mathbf{6} & 0 & 0 & 0 & 0 & 0 & 1 \end{array}$$

Network - Measures

Measures:

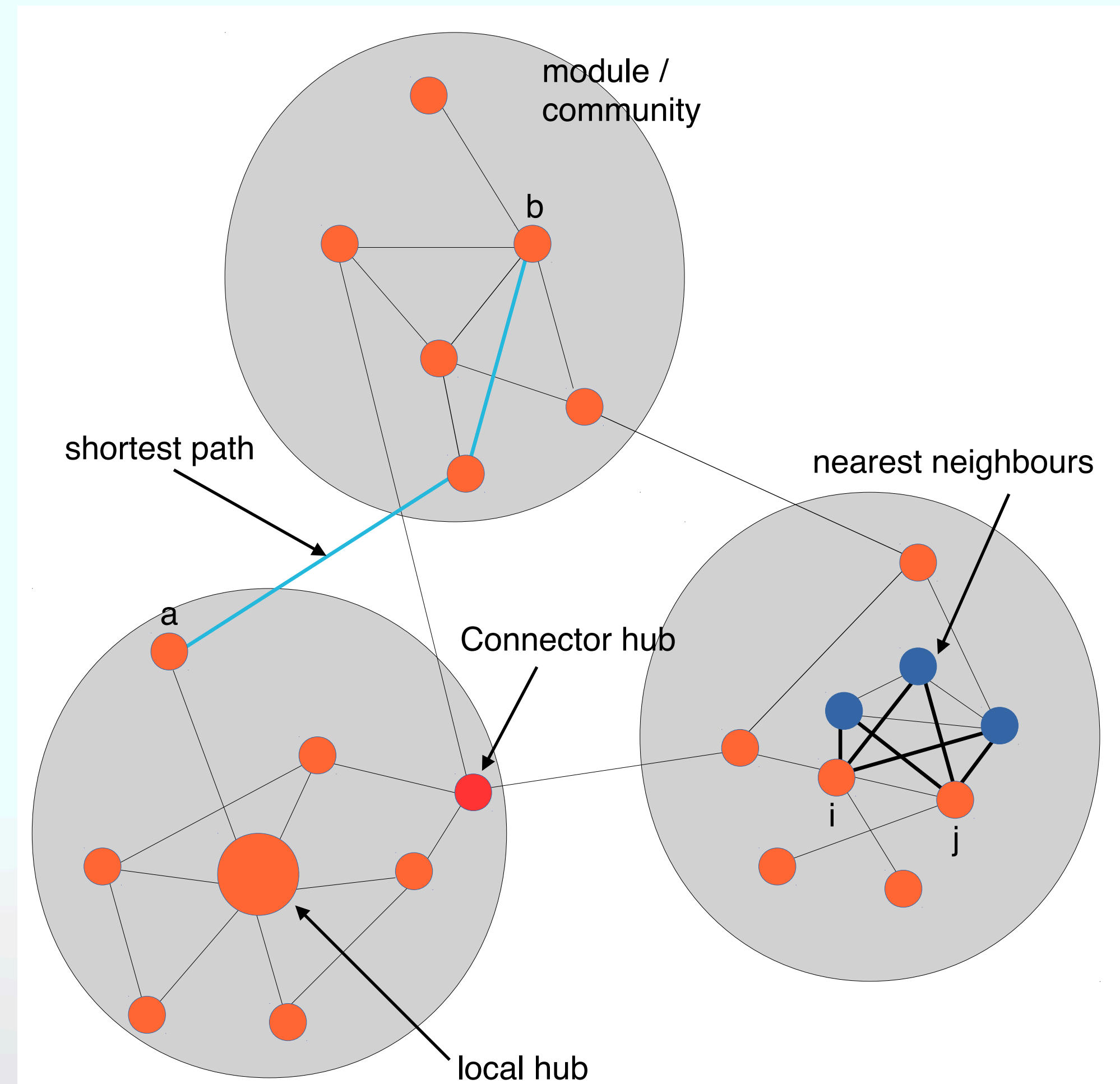
- From our mathematical representation, we can define many concepts which reveal a network's underlying character.

Centrality:

- "What are the most important or central nodes (i.e. influencer, disease hub) in our network?"
- Degree, Betweenness, Google's PageRank, Semi-local, Closeness...

Similarity:

- "How similar two nodes are to each other."
- how many shared neighbours.
- Pearson correlation, Modularity.



Clustering Networks - Community Clustering

General Approach

Graph Representation

- represent the data as a graph, where nodes represent entities and edges represent connections or relationships between them

Initialise Clusters

- start with each node as its own individual cluster or use an initial partitioning method to assign nodes to initial clusters

Calculate Modularity

- compute the initial modularity score, which quantifies the density of connections within clusters compared to connections between clusters

Optimise Modularity Locally

- iteratively merge or reassign nodes or small groups of nodes to maximise modularity
- for each iteration, assess the change in modularity resulting from moves or merges

Evaluate Modularity Gain

- accept moves or merges that improve the modularity score, and reject those that do not

Repeat Until Convergence

- continue optimising the modularity score until no further significant improvements can be made

Refine Clusters

- optionally, reapply the process at different resolutions (e.g., hierarchical clustering) to refine the clusters

Output Final Clusters

- when modularity optimisation reaches a stable maximum, finalise and output the clusters

Parameters

Resolution Parameter

- adjusts the size of the communities detected
- higher values of gamma often yield smaller localised communities, lower values yield larger, broader communities
- important for tuning clusters to the desired level of granularity

Initial Partitioning (Starting Clusters)

- determines the starting point for the modularity optimisation process
- some methods start with each node as its own cluster, while others use pre-defined partitions to initialise the process
- can affect the convergence and final clustering results

Edge Weights

- defines the weight or strength of the connections between nodes, where available
- higher weights indicate stronger connections, which can increase likelihood of nodes staying in the same community
- edge weights are particularly useful for networks with varying strengths of relationships, such as social networks

Maximum Iterations

- sets a limit on the number of iterations for the optimisation algorithm
- ensures that the algorithm does not run indefinitely and helps manage computational cost

Stopping Criterion (Convergence Threshold)

- specifies the minimum change in modularity required to continue iterations
- when the change in modularity between iterations falls below this threshold, the algorithm stops

Random Seed (for stochastic methods)

- controls the random initialisation for algorithms that have a stochastic component, such as the Louvain method
- ensuring a consistent random seed can make results reproducible

Algorithm Type

- choice of modularity-based algorithm, such as Louvain, Leiden, or Girvan-Newman
- different algorithms have unique approaches to modularity optimisation, affecting speed, accuracy, and scalability

Modularity Function Type

- specifies the formula for modularity, as there can be variations
- the choice depends on the network structure and the specific goals of the clustering

Clustering Networks - Hierarchical Clustering

agglomerative - (bottom-up)

General Approach

Initialise clusters

- treat each data point as its own cluster (single cluster for each data point)

Compute initial distances

- calculate the distance between every pair of clusters
- with average linkage, the distance between clusters is the average distance between each point in one cluster and each point in the other

Merge closest clusters

- identify the two clusters with the smallest average linkage distance and merge them to form a new cluster

Update distances

- recalculate the distances between the new cluster and all remaining clusters using average linkage

Repeat merging

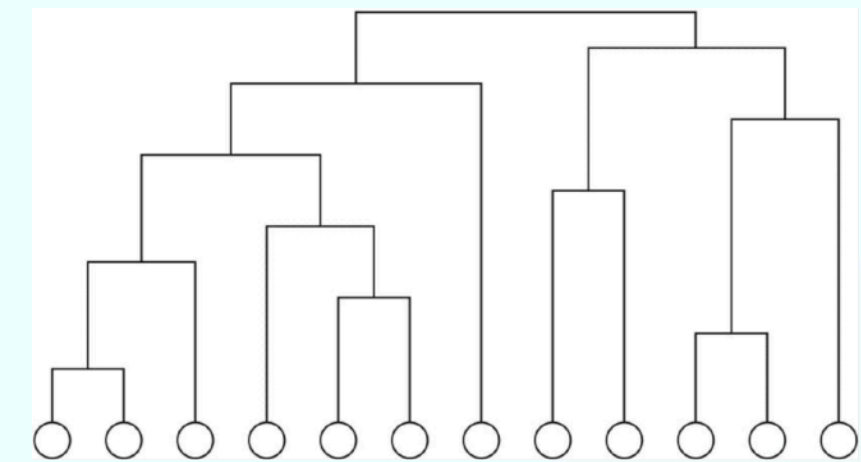
- continue merging the closest clusters and updating distances until only one cluster remains or until a desired number of clusters is reached

Build dendrogram

- track the order of merges to create a dendrogram, a tree-like structure that shows the hierarchical relationships between clusters

Determine clusters

- based on the dendrogram, decide on the final clustering by “cutting” the dendrogram at the desired level



classic hierarchical clustering

Distance Metrics

Euclidean Distance

- measures the straight-line distance between two points in Euclidean space
- commonly used in continuous, numeric data

Manhattan Distance

- computes the sum of absolute differences between points
- useful when dealing with high-dimensional data

Cosine Similarity

- measures the cosine of the angle between two non-zero vectors, indicating how aligned they are
- often used in text analysis and high-dimensional spaces

Jaccard Similarity

- measures the proportion of common elements in two sets over the union of the sets.
- commonly used for binary and categorical data, especially in set-based data.

Pearson Correlation

- measures the linear relationship between two variables, ranging from -1 to 1
- useful for continuous data where the focus is on correlation rather than direct distance

Distance

- counts the number of positions at which two binary strings differ
- ideal for categorical or binary data, such as in DNA sequence analysis

Over Representation Analysis (ORA)

- consideration of foreground and background lists is crucial
 - genome ?
 - array content ?
 - mappable elements ?

contingency table	foreground	background	Row Total
genes with Term in list	<i>a</i> 10	<i>b</i> 50	60
genes with Term not in list	<i>c</i> 190	<i>d</i> 13950	14140
Column Total	200	14000	<i>n</i> 14200

$$p = \frac{\binom{a+b}{a} \binom{c+d}{c}}{\binom{n}{a+c}} = \frac{(a+b)! (c+d)! (a+c)! (b+d)!}{a! b! c! d! n!}$$

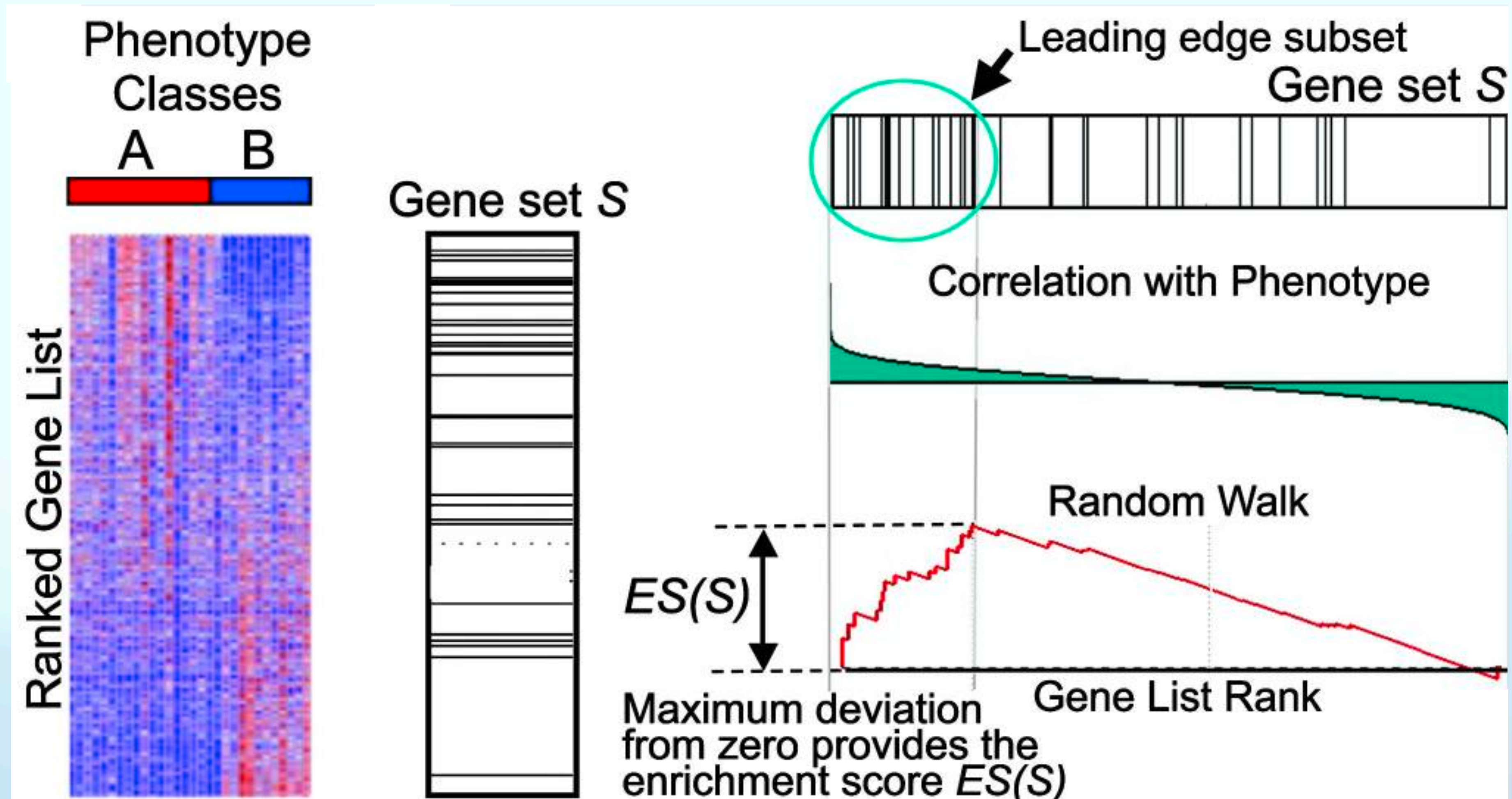
probability given by the hypergeometric distribution

in order to calculate significance we need to

- one-tailed test
sum all probabilities as extreme or more extreme
- two-tailed test
sum as above but also all that are as extreme or more extreme in both directions

p-value = 1.003x10⁻⁸ odds ratio = 14.68

Gene Set Enrichment Analysis (GSEA)



Ontologies or Terminologies

Terminologies

Controlled Vocabulary

defined list of terms that are used consistently within a specific domain, ensuring standardised naming.

Synonyms and Lexical Variants

provides different names or terms that refer to the same concept, enabling matching of varied terms to the same meaning.

Hierarchical Structure

terms may be organised in a hierarchy (e.g., disease categories) with broader and narrower terms to aid in understanding relationships.

Coding Systems

often assigned alphanumeric codes (e.g., ICD-10, SNOMED CT) for easy identification and reference.

Versioning and Updates

terminologies are periodically updated to reflect advances in medicine and changes in terminology.

Cross-Referencing with Other Terminologies

terms often have mappings to other terminologies, enabling data integration across systems (e.g., mapping ICD codes to SNOMED CT).

Ontologies

Formal Representation of Knowledge

define and formalise concepts, relationships, and rules within a domain, providing a deeper semantic structure than terminologies

Conceptual Hierarchies

often have structured hierarchies that describe relationships (e.g., "is-a," "part-of") among concepts, supporting logical inferences.

Logical Axioms and Rules

define rules and logical constraints for how concepts relate, enabling reasoning engines to deduce new information.

Interoperability

are designed for integration, allowing different systems to interpret and use the same data consistently

Linking and Reuse of Concepts

often link to other ontologies (e.g., linking human disease ontology with chemical ontology), promoting data integration across disciplines.

Defined Semantic Relationships

Unlike terminologies, ontologies provide explicit definitions of relationships between terms (e.g., "causes," "treats," "affects").

Semantic Annotation

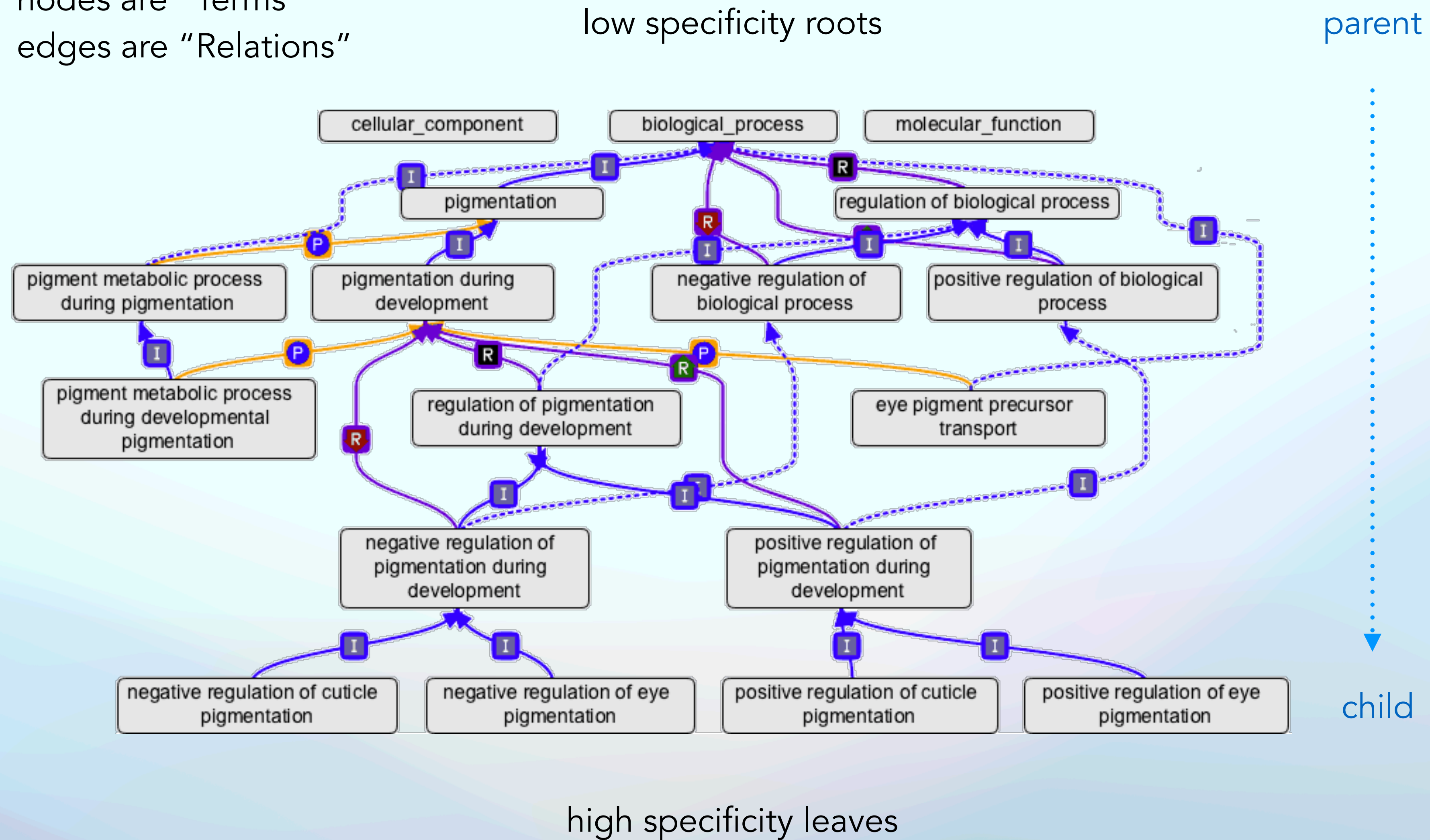
allow for tagging or annotating data with well-defined concepts, facilitating searches and analysis based on these annotations.

Inference Capabilities

support automated reasoning, enabling the system to infer new knowledge by applying rules to the defined relationships.

Structure of an Ontology

- nodes are "Terms"
- edges are "Relations"



Unified Medical Language System UMLS

<https://uts.nlm.nih.gov/>


National Library of Medicine

[Sign In](#)
[Sign Up](#)
[Contact Us](#)

UMLS Terminology Services
About
Browse
Download
APIs
Tools
Help

Welcome to UMLS Terminology Services (UTS). Your UTS account provides access to the Unified Medical Language System (UMLS), the Value Set Authority Center (VSAC), RxNorm downloads, SNOMED CT downloads and more.

Unified Medical Language System (UMLS)

A set of files and software that brings together many health and biomedical vocabularies and standards to enable interoperability between computer systems.

[Home](#) | [Browse](#) | [Download](#) | [API](#)

Value Set Authority Center (VSAC)

A repository and authoring tool for standard lists of codes and terms from biomedical vocabularies.

[Home](#) | [Browse](#) | [Download](#) | [API](#)

RxNorm

RxNorm provides normalized names for clinical drugs and links its names to many of the drug vocabularies commonly used in pharmacy management and drug interaction software.

[Home](#) | [Browse](#) | [Download](#) | [API](#)

SNOMED CT

SNOMED CT is one of a suite of designated standards for use in U.S. Federal Government systems for the electronic exchange of clinical health information.

[Home](#) | [Browse](#) | [Download](#)

UTS Profile

[Visit Your Profile](#)
[Generate an API Key](#)

Browse

[UMLS Metathesaurus](#)
[Value Set Authority Center](#)
[RxNorm](#)
[SNOMED CT](#)

Download

[UMLS](#)
[Value Sets](#)
[RxNorm](#)
[SNOMED CT](#)

Metathesaurus: Terms and codes from many vocabularies, including CPT, ICD-10-CM, LOINC, MeSH, RxNorm, and SNOMED CT. Hierarchies, definitions, and other relationships and attributes.

Semantic Network: Broad categories (semantic types) and their relationships (semantic relations).

SPECIALIST Lexicon and Lexical Tools: A large syntactic lexicon of biomedical and general English and tools for normalising strings, generating lexical variants, and creating indexes.

Current Release Statistics

Concepts: 3,426,877
concept names (AUIs): 16,709,195
concept names (SUIs): 13,775,220
normalised concept names (LUIs): 12,578,717
Number of sources: 170
Number of languages contributing concept names: 28

Opportunities & Challenges in Biomedical Informatics

Opportunities

Clinical & Health

- Administration Support
- Decision Support
- Patient Engagement
- Synthetic Data Generation
- Clinical Trial Design & Monitoring
- Population Level Modelling
- Professional Education

Biomedical Science

- Drug Discovery and Design
- Protein Structure Prediction
- Biomedical Image Synthesis
- Patient Data Generation
- Drug Response Prediction
- Biological Sequence Generation
- Medical Text Generation
- Biomedical Signal Generation
- Disease Progression Modeling

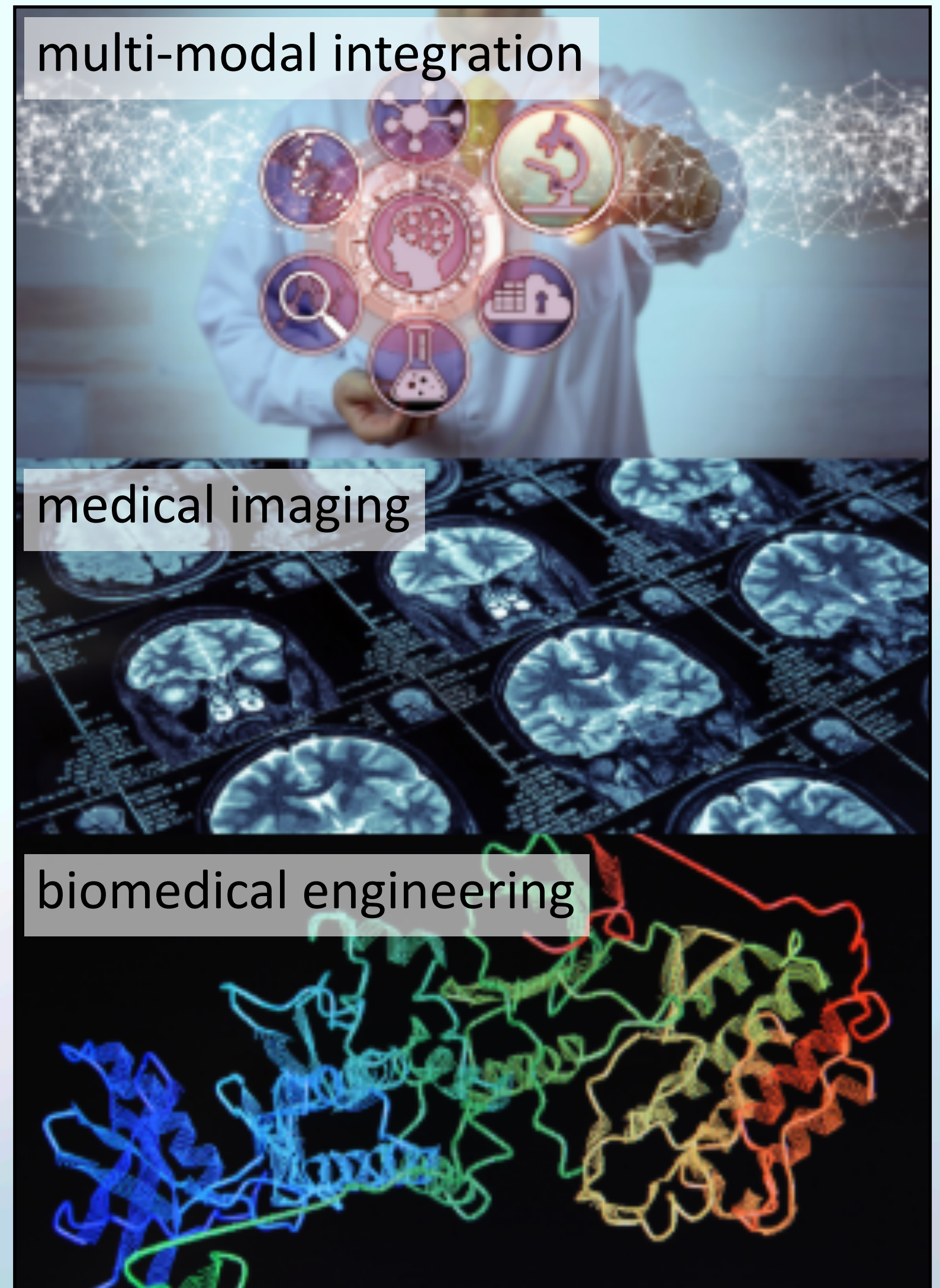
Challenges

Technical Challenges

- Unlabelled & Unstructured Data
- Missing Values
- Model & Data Bias
- Poor Longitudinal Coverage
- Scaling Problems
- Lack of Realistic Evaluation Benchmarks
- Explainability
- Data Availability & Inter-Operability

Societal & Health Systems

- Clinical safety, Efficacy, & Reliability
- Evaluation, Regulation, & Certification
- Privacy
- Copyright & Ownership
- Implementation & Adoption



Why Integrate Data?

Advantages

Holistic View of Biological Systems

Combines diverse data types (e.g., genomics, proteomics, and clinical data) to provide a comprehensive understanding of complex biological processes.

Improved Disease Mechanism Discovery

Links molecular-level data (e.g., gene expression) with phenotypic data (e.g., disease symptoms) to uncover disease pathways and biomarkers.

Enhanced Predictive Models

Increases the accuracy and robustness of predictive models by incorporating multi-dimensional data.

Cross-Validation of Findings

Enables verification of results across different datasets, increasing confidence in findings.

Identification of Novel Patterns

Facilitates the discovery of relationships and patterns not evident within individual datasets.

Personalised Medicine

Supports tailored healthcare approaches by integrating genomic, clinical, and lifestyle data for individualized treatment.

Efficient Use of Resources

Leverages existing datasets, reducing the need for redundant experiments.

Support for Multi-Scale Analysis

Bridges scales of biology, from molecular interactions to organ-level and population-wide studies.

Disadvantages

Data Heterogeneity

- Different data types (e.g., genomic vs. clinical) may have varying formats, scales, and resolutions, making integration complex.
- Integrated data may contain errors or missing values from individual sources, compounding the problem in the combined dataset.
- Lack of universal standards for data collection and annotation complicates integration across studies or institutions.
- Integrating data may introduce artificial patterns or average out real biological signal.

Complexity of Interpretation

Multi-modal data can produce results that are difficult to interpret, requiring domain expertise across multiple fields.

Data Availability and Access

Limited access to proprietary or sensitive datasets can restrict integration efforts.

Computational Challenges

Requires significant computational power and advanced algorithms to handle large, multi-dimensional datasets.

Bias

Integration may amplify biases inherent in individual datasets (e.g., population-specific biases).

Privacy and Ethical Concerns

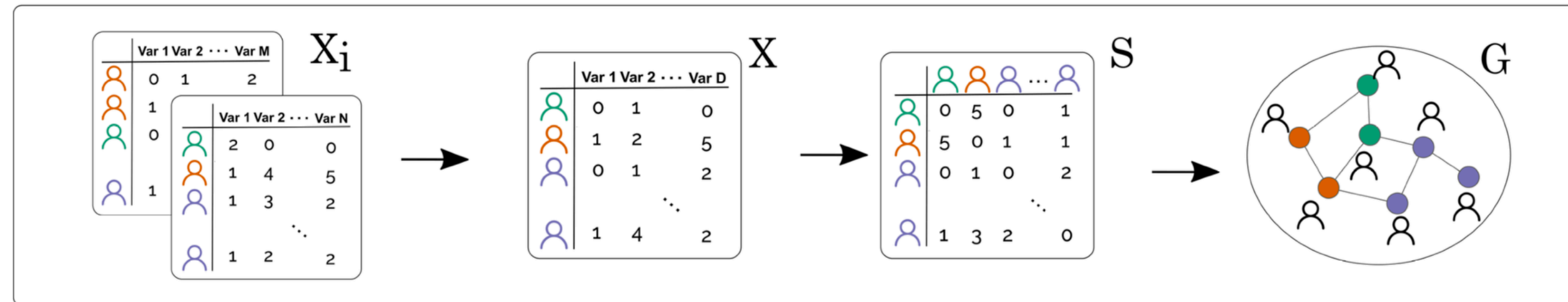
Combining sensitive data (e.g., genomic and clinical information) increases the risk of privacy breaches and ethical issues.

Cost and Resource Intensive

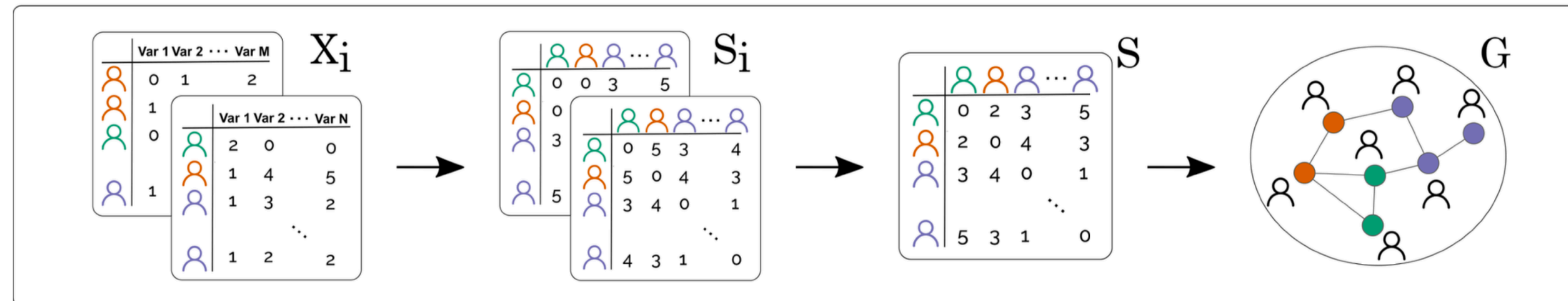
Collecting, processing, and integrating diverse datasets often requires significant financial and human resources.

Strategies for Multi-Modal Network Integration

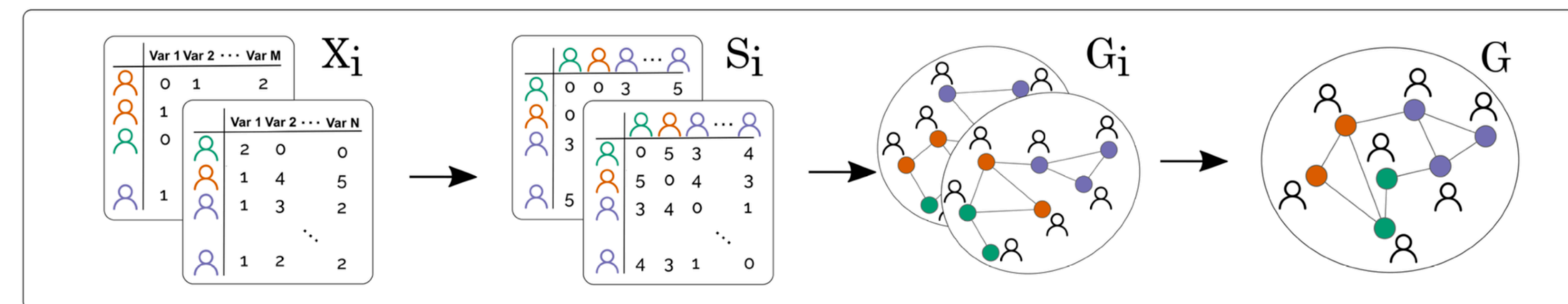
Early Integration



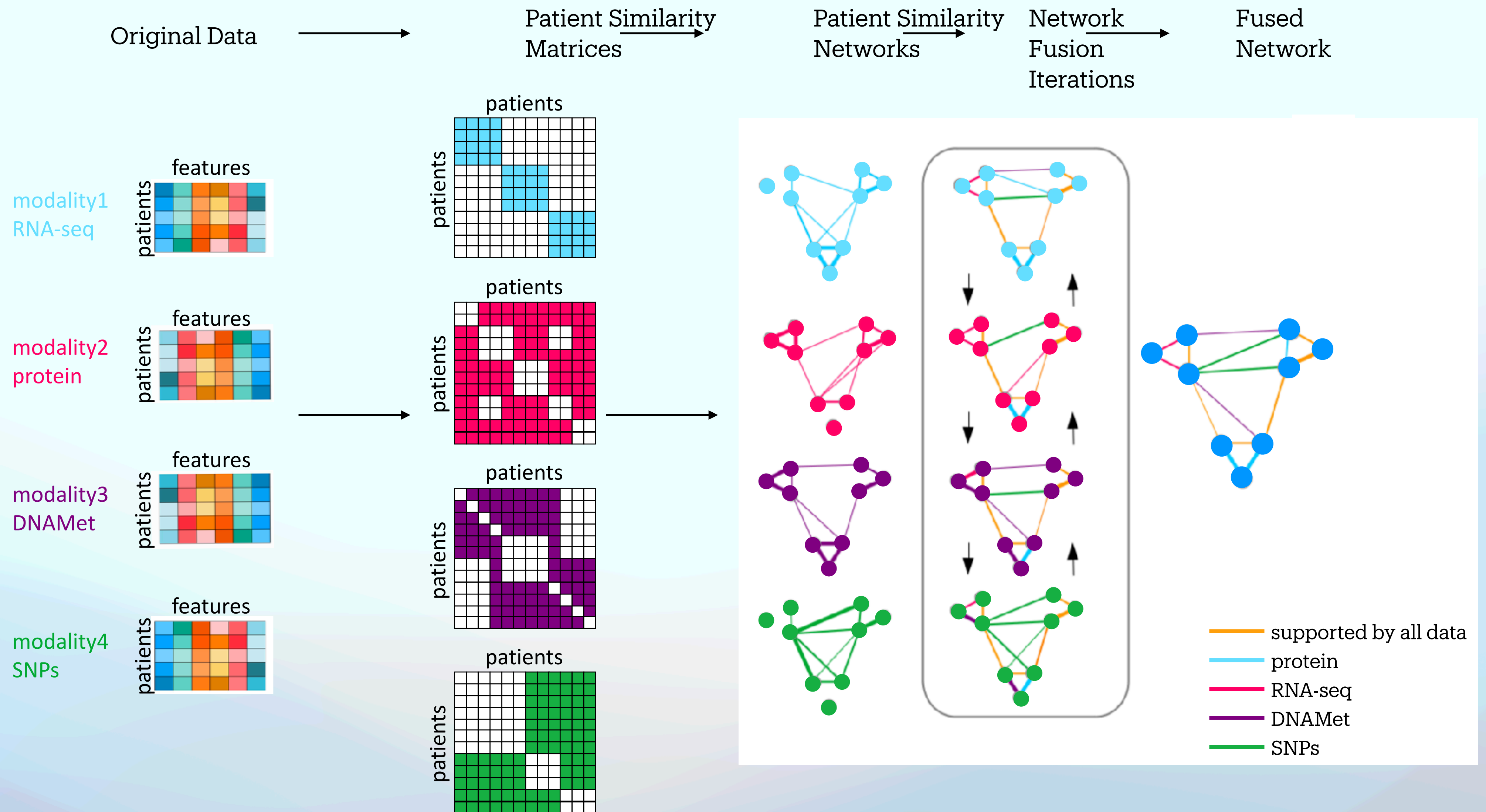
Intermediate Integration



Late Integration



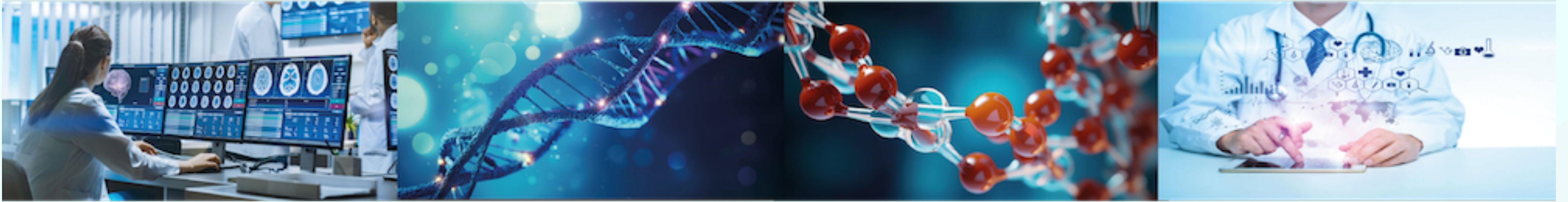
Multi-Modal Integration Through Similarity Network Fusion



after Wang, B. et al. Similarity network fusion for aggregating data types on a genomic scale. Nat Methods 11, 333–337 (2014).

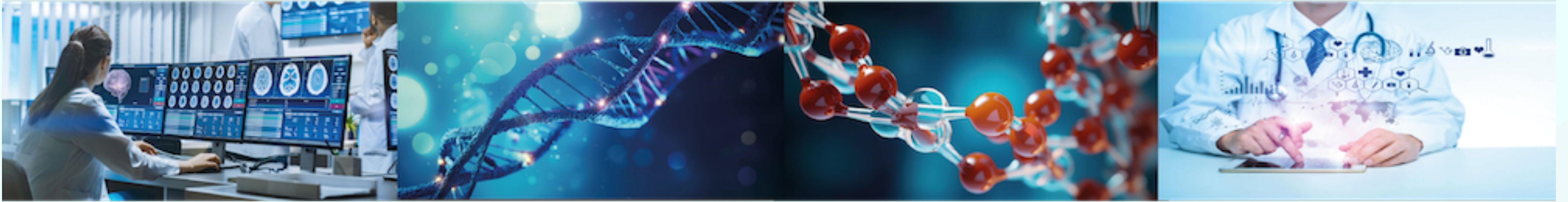
Example Short Answer Questions

1. Why might you consider Gene Set Enrichment Analysis (GSEA) to be a better approach to use than standard Over Representation Analysis (ORA)? (2 marks)
2. What does normalisation of gene expression data achieve? (2 marks)
3. Name two properties that ontologies have that make them well suited as reference structures for storing prior knowledge about biological systems? (2 marks)
4. What is the difference between the OBO-Foundry and the NCBO BioPortal? (2 marks)
5. What is BioMart? What methods can you use to access it programmatically? (2 marks)



“Guidance on Pseudo-coding and Final Q&A”

Content in the PsuedoCoding Notebook for week 11 on GitHub



Programming for Biomedical Informatics

Working up to the exam we will be monitoring the discussion board please do ask if you would like us to clarify and/or explain anything.

We Hope You Enjoyed the Course!